



Deep learning

I. Clasificación: Máquinas de Vector Soporte SVM

Tarea 1

Detección de fraudes en reclamación de seguros

Con el repositorio insurance_claims.csv que está disponible en el siguiente enlace:

<https://www.kaggle.com/datasets/aashishjhamtani/automobile-insurance>

Realice lo siguiente en un Notebook de Jupyter:

1. Cargue los datos en Python usando Pandas y muestre los primeros 10 registros.
2. Remplace '?' por np.NaN y verifique si hay valores nulos con el siguiente código:

```
data = data.replace('?', np.NaN)
data.isnull().any()
```

3. Remplace los valores faltantes de collision_type por la **moda** y los de las otras dos columnas por 'NO'.
4. Muestre las dimensiones del dataframe (data.shape) y el tipo de las columnas (data.info())
5. Muestre los valores estadísticos básicos de todas las columnas (contador, media, desviación estándar, valor mínimo, valor máximo y los cuartiles).
6. Muestre el mapa de calor de la matriz de correlación y su formato tidy.
7. Elimine las columnas que están altamente correlacionadas (>0.7) ignorando la variable objetivo: **fraud_reported**.

Revise el código del ejercicio de Propiedades de Brooklyn.

8. Elimine las columnas de alta cardinalidad **excepto** las siguientes columnas:

- | | |
|---------------------------|-----------------|
| • insured_education_level | • incident_date |
| • insured_occupation | • incident_city |
| • insured_hobbies | • auto_make |
| • insured_relationship | |

ya que estas columnas son muy importantes para el modelo.

9. Elimine las columnas de varianza baja (ninguna, pero realice la prueba).
10. Elimine la columna policy_number porque los identificadores no son de utilidad en los modelos de machine learning.



11. Con el siguiente código genere el gráfico de dispersión entre las columnas `total_claim_amount` y `policy_annual_premium`:

```
f, ax = plt.subplots(1, 1, figsize=(12, 6))
ax = sns.scatterplot(x='total_claim_amount', y='policy_annual_premium',
hue='insured_sex', data=data, palette='cool')
```

12. Genere el gráfico de barras de la variable objetivo:

```
ax = data['fraud_reported'].value_counts().plot(kind='bar', figsize=(8, 10),
fontsize=13, color=["red", "blue"])
ax.set_title('Fraude (Y = Fraude, N = No fraude)', size=20, pad=30)
ax.set_ylabel('Número de Fraudes', fontsize=10)

for i in ax.patches:
    ax.text(i.get_x() + 0.19, i.get_height() + 10, str(round(i.get_height(), 2)),
    fontsize=15)
```

13. Genere el gráfico boxenplot de la relación entre `incident_type` y `total_claim_amount`:

```
plt.style.use('fivethirtyeight')
plt.rcParams['figure.figsize'] = (10, 8)

sns.boxenplot(x=data['incident_type'], y=data['total_claim_amount'], palette =
'pink')
plt.title('Incident Type vs total_claim_amount', fontsize = 20)
plt.show()
```

14. Con el siguiente código genere el gráfico de tabulación cruzada entre las columnas `incident_city` e `incident_type`:

```
incident = pd.crosstab(data['incident_city'], data['incident_type'])
colors = plt.cm.summer(np.linspace(0, 1, 5))
incident.div(incident.sum(1).astype(float), axis = 0).plot(kind = 'bar',
stacked = True,
figsize = (15, 7),
color = colors)

plt.title('Incident city vs Incident Type', fontsize = 20)
plt.legend()
plt.show()
```



15. Genere los gráficos de tabulación cruzada entre las siguientes columnas:

- incident_type e incident_severity
- incident_type y collision_type
- insured_occupation y fraud_reported
- insured_hobbies y fraud_reported
- insured_education_level y fraud_reported

Pruebe con alguno de los siguientes valores:

- Paletas de colores: plt.cm.Blues, plt.cm.summer, plt.cm.inferno
- stacked = True,False

Puede consultar otras paletas de colores en el siguiente enlace:

<https://matplotlib.org/stable/users/explain/colors/colormaps.html>

16. Realice el siguiente grafico de distribución:

```
plt.style.use('fivethirtyeight')
plt.rcParams['figure.figsize'] = (15, 8)

sns.swarmplot(x=data['policy_state'], y=data['total_claim_amount'], palette =
'copper')
plt.title('Policy State vs Total Claim Amount', fontsize = 20)
plt.show()
```

17. Realice el siguiente gráfico de pastel de la relación de insured_sex con reporte de fraude

```
X=data.where(data['fraud_reported']=='Y')
a=pd.DataFrame(X['insured_sex'].value_counts())
a.reset_index(inplace=True)

a.plot(kind='pie',y='insured_sex',labels=a['index'],autopct='%1.2f')
plt.show()
```

Explicación:

Where filtra del dataframe aquellos registros que cumplen las condiciones indicadas, en este caso solo los registros de fraude.

18. Realice el grafico de pastel de la relación que existe entre el reporte de fraude con las siguientes columnas:

- insured_relationship
- insured_hobbies
- insured_occupation
- incident_type



19. Realice el siguiente diagrama de bigotes:

```
sns.boxplot(x='auto_make',y='total_claim_amount',data=data)
plt.show()
```

20. Realice el diagrama de bigotes de months_as_customer

21. Realice los diagramas de distribución con el siguiente código:

```
import seaborn as sns
import matplotlib.pyplot as plt

col=['months_as_customer','policy_deductable','policy_annual_premium','capital-
gains','capital-loss','total_claim_amount']
plt.figure(figsize=(18,21))
k=1
for i in col :
    plt.subplot(3,2,k)
    sns.distplot(data[i])
    k=k+1
plt.show()
```

22. Obtenga el día del accidente de la columna incident_date con el siguiente código:

```
def date_tonumberday(fecha):
    fecha = pd.to_datetime(fecha, errors = 'coerce')
    day_of_year = fecha.toordinal() - date(fecha.year, 1, 1).toordinal() + 1
    return day_of_year
data['incident_day']=data['incident_date'].apply(date_tonumberday)
data.drop(columns=['incident_date'],axis=1,inplace=True)
```

23. Codifique la variable objetivo con el siguiente código:

```
data['fraud_reported'] = data['fraud_reported'].replace(('N','Y'),(0,1))
data['fraud_reported'].head()
```

24. Haga lo mismo con las columnas: police_report_available, property_damage ('NO'=0, 'Y'=1), insured_sex ('FEMALE'=0, 'MALE'=1), incident_severity ("Trivial Damage"=0.1, "Minor Damage"=0.3, "Major Damage"=0.6, "Total Loss"=1.0), insured_education_level ('High School'=0.63, 'College'=0.7, 'Associate'=0.78, 'Masters'=0.92, 'MD'=0.92, 'JD'=0.92, 'PhD'=1.0)

Puede consultar detalles sobre los años de estudio en USA en el siguiente enlace:

<https://iicana.org/asesoria-educativa/sistema-educativo-de-estados-unidos/>



25. Codifique al formato One-hot la columna auto_make con el siguiente código:

```
cols = pd.get_dummies(data.auto_make)
data = pd.concat([data.drop("auto_make",axis="columns"), cols], axis="columns")
data.head()
```

26. Haga lo mismo con las columnas no ordinales: incident_city, policy_state, authorities_contacted, collision_type, incident_type, insured_relationship, insured_hobbies, insured_occupation y policy_csl.

27. Normalice las columnas con MinMaxScaler.

28. Separe las columnas independientes de la variable objetivo ('fraud_reported') en X,Y

29. Use la función train_test_split para separar el conjunto de datos en 80% entrenamiento y 20% prueba.

30. Haga una búsqueda de la mejor configuración de hiper parámetros con GridSearchCV para el Support Vector Machine con los siguientes parámetros:

```
param_grid = {'C':np.logspace(0, 2, 20), 'gamma':['scale',1/138,1/100,1/150,1/180]}
grid = GridSearchCV(
    estimator = SVC(kernel= "rbf"),
    param_grid = param_grid,
    scoring    = 'accuracy',
    n_jobs     = -1,
    cv         = 3,
    verbose    = 0,
    return_train_score = True
)
```

31. Muestre la mejor configuración encontrada.

32. Haga las predicciones del modelo con los datos de prueba y muestre la exactitud del modelo.

33. Muestre la matriz de confusión del modelo.



34. Indique las dimensiones de los vectores de soporte del modelo (modelo.support_vectors_.shape).

35. Repita los pasos 30-34 pero ahora con los siguientes parámetros:

Kernel 'linear'
 'C': [0.08, 0.09, 0.1, 0.11, 0.12]
 Mismo gamma

Kernel polinomial 'poly'
 'C': [500, 550, 600, 650, 700, 750]
 'degree': [3, 4, 5]
 Mismo gamma

Kernel sigmoide 'sigmoid'
 'C': np.logspace(0, 2, 20)
 Mismo gamma

36. En un Excel que adjunte o una tabla html de jupyter-lab registre los mejores resultados de cada configuración.

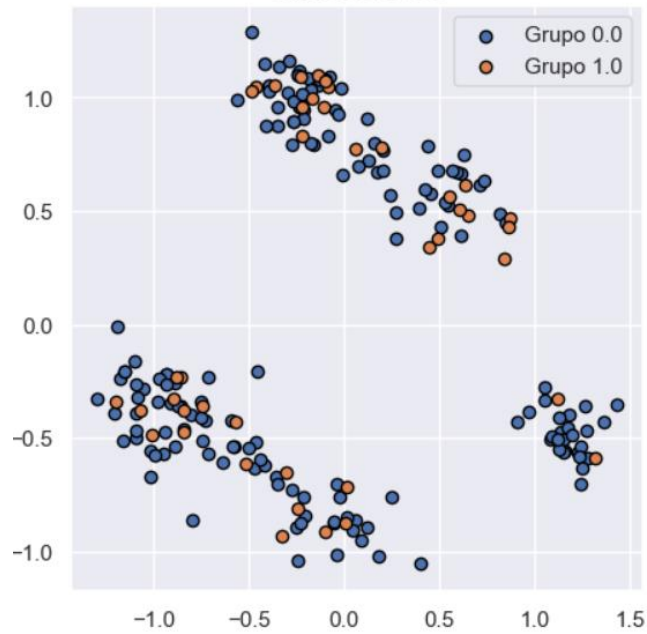
Kernel	Valor C	Valor Gamma	% exactitud	# vectores soporte
Radial				
Lineal				
Polinomial				
Sigmoide				

Como recordara cuando aplicamos arboles de decisión logramos un 75.5% de exactitud, pero aplicando Random Forest se consigue hasta un 87%, sin embargo, recuerde que esos modelos tenían la desventaja de que si los datos se salían del rango estudiado tenían un mal desempeño por lo que el mejor modelo dependerá siempre del caso de estudio y de las restricciones que se presenten.

Podría aplicar componentes principales para graficar la distribución de las muestras e indicar visualmente los aciertos obteniendo el siguiente grafico:



Etiquetas reales



Clasificación generada con SVM

